LAB 7

2. Higher-Order Functions: List.map, List.filter, and List.fold (Exp 12)

Write F# programs demonstrating three higher-order functions using pure lambda expressions:

(1) List.map — transform each element (e.g., square all integers);

(2) List.filter — select elements satisfying a predicate (e.g., retain only even numbers);

(3) List.fold or List.reduce — aggregate a list (e.g., compute sum, product, or maximum).

(4) Test each on at least two different lists and print results.



```
printfn "HIGHER ORDER FUNCTIONS IN F#"

let list1 = [1; 2; 3; 4; 5]
let list2 = [6; 7; 8; 9; 10]

let square1 = List.map (fun x -> x * x) list1
let square2 = List.map (fun x -> x * x) list2

printfn "\nList.map:"
printfn "%A" square1
printfn "%A" square2

let even1 = List.filter (fun x -> x % 2 = 0) list1
let even2 = List.filter (fun x -> x % 2 = 0) list2

printfn "\nList.filter:"
printfn "%A" even1
printfn "%A" even2

let sum1 = List.fold (fun acc x -> acc + x) 0 list1
let sum2 = List.fold (fun acc x -> acc + x) 0 list2

printfn "\nList.fold:"
printfn "Sum of List 1 = %d" sum1
printfn "Sum of List 2 = %d" sum2
```



```
C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\EXP12_HigherOrder>dotnet run
HIGHER ORDER FUNCTIONS IN F#

List.map:
[1; 4; 9; 16; 25]
[36; 49; 64; 81; 100]

List.filter:
[2; 4]
[6; 8; 10]

List.fold:
Sum of List 1 = 15
Sum of List 2 = 40

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\EXP12_HigherOrder>
```